

Stack Investor: Stock Signal Classification Using Technical Indicators and News Sentiment

Project Final Report

Team Members: Erroll Barker, Prakeerth Regunath, Bae Meh

Abstract

This project studies next-day stock signal classification using supervised machine learning. Our goal is to predict **Buy**, **Hold**, or **Sell** signals for a set of large-cap U.S. stocks using two types of information: technical indicators from daily OHLCV price data and aggregated daily news sentiment from financial headlines. We compare a technical-only pipeline with a technical-plus-sentiment pipeline and evaluate multiple models, including a Holdout baseline, Logistic Regression, and XGBoost. The main historical experiment shows that XGBoost with technical and sentiment features performs best, achieving the strongest accuracy and Macro F1 among the tested models. In addition to the historical experiments, we extended the project into a live application prototype using Streamlit. The live prototype supports saved-model inference, live technical-only predictions, live headline ingestion from RSS feeds, and live daily sentiment aggregation. Overall, the project shows that sentiment can improve stock signal prediction, especially when paired with a nonlinear model such as XGBoost.

1 Introduction

Predicting stock market movement is a challenging machine learning problem because financial time series are noisy, highly variable, and influenced by both numerical market behavior and unstructured external information such as news. Traditional rule-based strategies, for example, using only one technical indicator such as RSI or a moving average crossover, can be useful in limited settings but are often too rigid to generalize across changing market conditions. This makes stock signal prediction a natural application for machine learning, where a model can combine multiple inputs and learn patterns from historical data.

In this project, we formulate stock prediction as a supervised three-class classification problem. For each stock-day example, the goal is to predict whether the next-day movement should be interpreted as **Buy**, **Hold**, or **Sell**. We focus on a set of large-cap U.S. stocks and build feature vectors from daily OHLCV market data. We then extend the feature space by incorporating daily aggregated sentiment derived from news headlines. This allows us to test whether sentiment contributes useful predictive information beyond price-based technical indicators alone.

The project has two major parts. The first part is the historical machine learning study, where we compare a technical-only pipeline against a technical-plus-sentiment pipeline using chronological train, validation, and test splits. The second part is a live application prototype that extends the original project into a usable dashboard. The live version supports saved-model inference, current technical-only predictions, RSS-based live news ingestion, and live daily sentiment aggregation. This extension moves the project beyond a standard course experiment and toward a functioning applied system.

Our main models are Logistic Regression and XGBoost. Logistic Regression serves as a strong linear baseline, while XGBoost serves as the main nonlinear model because it performs well on structured tabular data. We also use a trivial Hold baseline for comparison. The central hypothesis of the project is that sentiment should improve predictive performance, especially for a nonlinear model that can capture interactions between technical indicators and sentiment features.

2 Related Work

A major algorithmic reference for this project is XGBoost, introduced by Chen and Guestrin. XGBoost is a scalable gradient-boosting framework that has become a strong standard for tabular machine learn-

ing tasks. Since our project relies on engineered numeric features such as returns, volatility measures, moving averages, RSI, MACD, and aggregated sentiment statistics, XGBoost is a natural choice. It is particularly attractive because it can model nonlinear feature interactions more effectively than simpler linear classifiers.

Another important area of related work is financial sentiment analysis. General sentiment tools often do not capture the tone and wording of financial texts well, which motivates the use of domain-adapted sentiment models. Work such as FinBERT demonstrates that financial text sentiment can be useful for downstream tasks and that finance-specific text modeling can outperform generic sentiment methods. Our project does not implement a transformer-based finance model, but it follows the same general idea that text-derived sentiment can provide useful information for market-related prediction tasks.

There is also substantial prior work on combining market indicators with text or news information for stock movement prediction. These studies motivate our main ablation experiment: comparing a technical-only model against a technical-plus-sentiment model. Our project is similar in spirit to that literature because we combine structured market features with aggregated news sentiment. However, our implementation uses daily market-wide sentiment rather than ticker-specific headline alignment, which makes the setup simpler but also introduces an important limitation. In this sense, our work is best viewed as a practical, interpretable course project that tests whether sentiment improves performance in a realistic yet simplified pipeline.

Finally, our live application extension connects this project to applied machine learning system design. While many course projects stop at static experiments, our system addresses additional challenges: saved-model inference, live feature generation, data ingestion, and interface design. This live extension is not a full production system, but it helps illustrate how a historical ML experiment can be turned into an interactive prototype.

3 Methods

3.1 Problem Formulation

We formulate the task as a three-class supervised classification problem. Given a stock-day observation with engineered features, the model predicts one of three labels: **Sell**, **Hold**, or **Buy**. The label is constructed from the next-day return:

$$r_{t+1} = \frac{Close_{t+1} - Close_t}{Close_t}.$$

If $r_{t+1} > 0.003$, the example is labeled **Buy**. If $r_{t+1} < -0.003$, the example is labeled **Sell**. Otherwise, it is labeled **Hold**. This thresholding reduces noise by preventing extremely small daily changes from being treated as strong trading signals.

3.2 Dataset Collection

The project uses two main data sources. The first source is daily OHLCV stock data from Yahoo Finance. We use ten large-cap U.S. stocks:

AAPL, MSFT, AMZN, GOOG, META, NVDA, TSLA, JPM, XOM, NFLX.

These stocks provide sufficient historical price and volume data for generating technical features.

The second source is a dated news headline dataset. For historical experiments, we use the Kaggle RedditNews dataset and aggregate headlines by date into daily sentiment features. For the live prototype, we ingest headlines from RSS feeds and build a similar daily aggregated sentiment representation. This keeps the feature format consistent between the historical and live pipelines, even though the live version is still in a warm-up stage.

3.3 Data Preprocessing

For stock data, we download daily OHLCV prices, normalize column names, and align observations by ticker and date. For each ticker, the data is sorted chronologically. Rows with invalid or missing values are removed. Since technical indicators rely on rolling windows, the earliest rows are naturally dropped after feature construction.

For headline data, text is cleaned by removing line breaks and extra whitespace. Each headline is assigned a sentiment score using VADER. We then aggregate headlines by day to obtain a single daily sentiment summary. Because using the same-day sentiment directly could introduce leakage, the historical pipeline shifts sentiment by one day before merging with stock features. This means sentiment from day t is used as part of the feature vector for predicting the signal associated with day $t + 1$.

3.4 Feature Engineering

3.4.1 Technical Features

The technical feature set includes:

- 1-day, 5-day, and 10-day returns
- simple moving averages over multiple windows
- exponential moving averages over multiple windows
- ratios of price relative to moving averages
- moving-average crossover indicators
- rolling volatility measures
- intraday high-low range
- close-open gap
- RSI(14)
- MACD, MACD signal, and MACD histogram
- Bollinger Band features
- volume change and rolling volume averages

These features provide a compact summary of short-term trend, momentum, volatility, and trading activity.

3.4.2 Sentiment Features

The sentiment feature set includes:

- mean daily sentiment
- sentiment standard deviation
- minimum daily sentiment
- maximum daily sentiment
- average negative sentiment score

- average neutral sentiment score
- average positive sentiment score
- total headline count
- positive headline count
- negative headline count
- neutral headline count

This representation allows the model to capture both overall tone and headline volume. However, in the current implementation, these sentiment summaries are market-wide rather than ticker-specific.

3.5 Models

We evaluate three main models:

Hold Baseline. This trivial baseline always predicts **Hold**. It establishes the minimum meaningful performance level for the classification task.

Logistic Regression. Logistic Regression serves as a strong linear baseline. It is trained with standardized features and balanced class weighting. This model is easy to interpret and provides a natural comparison point for determining whether nonlinear modeling helps.

XGBoost. XGBoost is the main model used in this project. It is trained using a validation split for hyperparameter tuning, balanced sample weights, and early stopping. We additionally tune Buy and Sell decision thresholds to improve overall balanced classification performance.

3.6 Train/Validation/Test Splits

Because stock data is time-series data, we do not randomly shuffle examples. Instead, we use chronological splitting. The first 70% of dates are used for training, the next 15% for validation, and the final 15% for testing.

For the technical-only live training setup, the final dataset contained 24,630 rows, split into 17,240 training samples, 3,690 validation samples, and 3,700 test samples. For the historical technical-plus-sentiment experiment, the overlapping stock-news dataset contained 17,145 rows, split into 11,625 training, 2,760 validation, and 2,760 test samples.

3.7 Evaluation Metrics

We evaluate the models using:

- Accuracy
- Macro F1
- Weighted F1

Macro F1 is especially important because the classes are imbalanced, with the Hold class being smaller than the Buy and Sell classes. We also analyze class-level precision, recall, and F1 scores to better understand which models favor which classes.

3.8 Live Application Extension

After building the historical ML pipeline, we extended the project into a live application prototype using Streamlit. This required separating offline model training from live inference. The live prototype includes:

- saved XGBoost models and feature manifests
- live technical-only prediction using recent Yahoo Finance data
- a Streamlit dashboard for displaying predictions
- RSS-based headline ingestion
- live daily sentiment aggregation
- a sentiment mode that is structurally wired in but still warming up due to limited aligned live history

This extension demonstrates how a historical ML experiment can be transformed into an interactive application.

4 Experimental Results

4.1 Historical Model Comparison

The main historical experiment compares technical-only features against technical-plus-sentiment features. The results are shown in Table 1.

Experiment	Model	Accuracy	Macro F1	Weighted F1
Technical only	Hold baseline	0.1797	0.1016	0.0548
Technical only	Logistic Regression	0.3112	0.3101	0.3086
Technical only	XGBoost	0.3547	0.3478	0.3601
Technical + Sentiment	Hold baseline	0.1797	0.1016	0.0548
Technical + Sentiment	Logistic Regression	0.3116	0.3114	0.3103
Technical + Sentiment	XGBoost	0.3775	0.3720	0.3847

Table 1: Historical comparison of technical-only and technical-plus-sentiment models.

These results show several important patterns. First, both learned models outperform the trivial Hold baseline by a large margin. This confirms that the classification task is nontrivial and that the engineered features contain useful predictive information.

Second, Logistic Regression provides a strong baseline, but its improvement from adding sentiment is very small. Accuracy improves only from 0.3112 to 0.3116, and Macro F1 improves only from 0.3101 to 0.3114. This suggests that the linear model is not able to effectively exploit the interaction between technical indicators and sentiment.

Third, XGBoost performs best overall. In the technical-only setting, it achieves 0.3547 accuracy and 0.3478 Macro F1. After adding sentiment features, the accuracy improves further to 0.3775 and the Macro F1 to 0.3720. This is the strongest result in the project. The most reasonable interpretation is that sentiment contains useful information. Still, the relationship between market indicators and text-derived sentiment is nonlinear, which allows XGBoost to benefit more than Logistic Regression.

4.2 Class-Level Behavior

The class-level classification reports also show that **Hold** is the hardest class to model well. In the best-performing technical-plus-sentiment XGBoost configuration, the Hold class achieved lower precision than Buy or Sell, but it still maintained meaningful recall. This is important because the Hold class is both the minority class and the least directional class. In practice, this means the model is better at identifying directional moves than at confidently deciding when there is no strong action signal.

This class imbalance justifies using Macro F1 rather than relying solely on accuracy. A model that overpredicts the most common directional classes could still appear strong on raw accuracy while performing poorly on balanced classification quality. By reporting Macro F1, we emphasize balanced performance across all three labels.

4.3 Technical-Only Live Model

For the live version of the system, we trained and saved a technical-only XGBoost model over a broader historical stock-only window. This model achieved:

- Accuracy: 0.3486
- Macro F1: 0.3421
- Weighted F1: 0.3502

Although these values are slightly lower than the best historical technical-plus-sentiment model, the technical-only live model is the most stable real-time deployment path because it relies solely on current market data rather than a still-developing live sentiment history.

4.4 Live App Prototype Results

The Streamlit prototype successfully performs live technical-only inference. It loads a saved model, downloads recent stock data, builds the required technical feature set, and outputs current Buy/Hold/Sell predictions for the supported tickers. The interface also displays a selected ticker’s recent price chart, signal probabilities, and live pipeline status.

The live news ingestion stage is also functioning. Headlines are collected from RSS feeds and stored in a raw headline file. A second stage converts these headlines into a daily sentiment CSV file with the same schema as the historical experiments. The app also keeps a live status JSON file so the interface can show how many headlines have been collected and how many effective sentiment days are currently available.

However, the sentiment-based live mode remains in a warm-up state. The current issue is not a code failure but rather a lack of sufficient accumulated live headline history aligned with trading dates, under the same anti-leakage logic used in the historical experiments. Since historical sentiment is shifted by 1 day before prediction, the live system needs multiple aligned effective sentiment days before the sentiment model can reliably produce real-time outputs. As a result, the technical-only mode is currently the stable live demonstration path. In contrast, the sentiment mode is best described as structurally complete but still collecting enough usable live history.

4.5 Discussion

Overall, the experiments support three main conclusions. First, technical indicators alone are sufficient to build a meaningful stock signal classifier. Second, sentiment can improve predictive performance, but the benefit is much stronger for XGBoost than for Logistic Regression. Third, turning a historical ML project into a live application introduces new engineering challenges such as saved-model inference, live feature generation, and data alignment. These challenges do not invalidate the machine learning results, but they do affect what can be reliably deployed in real time.

5 Conclusion

In this project, we built a supervised machine learning system for classifying next-day Buy/Hold/Sell stock signals. We compared technical-only features against technical-plus-sentiment features and evaluated Hold baseline, Logistic Regression, and XGBoost models. Our best historical result came from XGBoost with technical and sentiment features, which outperformed the baselines on both accuracy and Macro F1. This supports the idea that sentiment can add useful predictive information beyond technical indicators alone, especially when paired with a nonlinear model.

We also extended the original course project into a live Streamlit application. The live system currently supports saved-model inference, technical-only real-time predictions, RSS-based news ingestion, and live sentiment aggregation. The main remaining limitation is that the live sentiment mode still needs more accumulated trading-day-aligned history before it can produce stable sentiment-based predictions. In future work, we would like to improve live sentiment alignment, use ticker-specific news rather than market-wide sentiment, add scheduled updates, and further improve the system’s deployment readiness.

6 References

1. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
2. D. Araci, “FinBERT: Financial Sentiment Analysis with Pre-trained Language Models,” 2019.
3. Yahoo Finance historical stock price data, accessed through the `yfinance` Python library.
4. Kaggle RedditNews dataset for dated news headlines used in the historical sentiment experiments.
5. VADER SentimentIntensityAnalyzer, used for headline sentiment scoring.
6. Streamlit, used for the live dashboard prototype.